

DC Experiments

Exp1:

Code : Create 2 Files: 1) Server.py
2) Client.py

Server.py file code:

```
from xmlrpc.server import SimpleXMLRPCServer
import math

def factorial(n):
    return math.factorial(n)

server = SimpleXMLRPCServer(("localhost", 9000))
print("Server running on port 8000...")

server.register_function(factorial, "factorial")
server.serve_forever()
```

Client.py file code :

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:9000/")

num = int(input("Enter number: "))
result = proxy.factorial(num)

print("Factorial:", result)
```

```
server.py  client.py x
1  import xmlrpc.client
2
3  proxy = xmlrpc.client.ServerProxy("http://localhost:9000/")
4
5  num = int(input("Enter number: "))
6  result = proxy.factorial(num)
7
8  print("Factorial:", result)
```

Run server x client x

"E:\Users\sanke\PycharmProjects\sanketproject\DC Exp-1\.venv\Scripts\python.
Enter number: 5
Factorial: 120

```
1  from xmlrpc.server import SimpleXMLRPCServer
2  import math
3
4  def factorial(n):
5      return math.factorial(n)
6
7  server = SimpleXMLRPCServer(("localhost", 9000))
8  print("Server running on port 8000...")
9
10 server.register_function(factorial, name="factorial")
11 server.serve_forever()
```

server x client x

"E:\Users\sanke\PycharmProjects\sanketproject\DC Exp-1\.venv\Scripts\python.exe" "E:\Users\sanke\...
Server running on port 8000...
127.0.0.1 - - [04/Apr/2026:20:15:18] "POST / HTTP/1.1" 200

Exp 2:

Server and Client Code in 1 File:

```
import Pyro5.api
```

Sever

```
@Pyro5.api.expose
```

```
class StringService:
```

```
    def concat(self, a, b):
```

```
        return a + b
```

```
daemon = Pyro5.api.Daemon()
```

```
uri = daemon.register(StringService)
```

```
print("URI:", uri)
```

```
daemon.requestLoop()
```

#Client

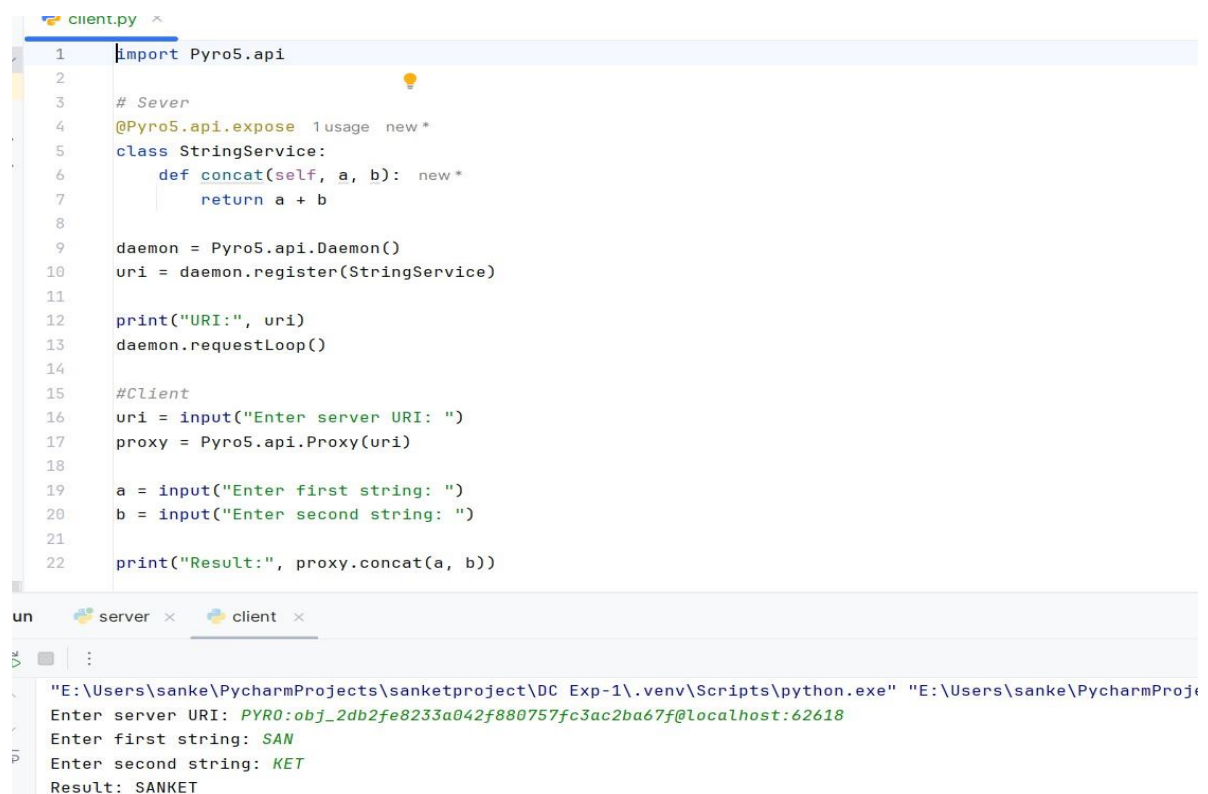
```
uri = input("Enter server URI: ")
```

```
proxy = Pyro5.api.Proxy(uri)
```

```
a = input("Enter first string: ")
```

```
b = input("Enter second string: ")
```

```
print("Result:", proxy.concat(a, b))
```



The screenshot shows a PyCharm IDE with a file named `client.py` open. The code in the file is a single script that can act as either a server or a client. It imports `Pyro5.api` and defines a `StringService` class with a `concat` method. It then creates a `Pyro5.api.Daemon` object, registers the `StringService` class, and prints the URI. Finally, it enters a `requestLoop`. The client part of the code prompts the user for a server URI, a first string, and a second string, and then prints the result of the `concat` method. The output window at the bottom shows the execution of the script, displaying the URI `PYRO:obj_2db2fe8233a042f880757fc3ac2ba67f@localhost:62618` and the result `SANKET`.

```
1 import Pyro5.api
2
3 # Sever
4 @Pyro5.api.expose
5 class StringService:
6     def concat(self, a, b):
7         return a + b
8
9 daemon = Pyro5.api.Daemon()
10 uri = daemon.register(StringService)
11
12 print("URI:", uri)
13 daemon.requestLoop()
14
15 #Client
16 uri = input("Enter server URI: ")
17 proxy = Pyro5.api.Proxy(uri)
18
19 a = input("Enter first string: ")
20 b = input("Enter second string: ")
21
22 print("Result:", proxy.concat(a, b))
```

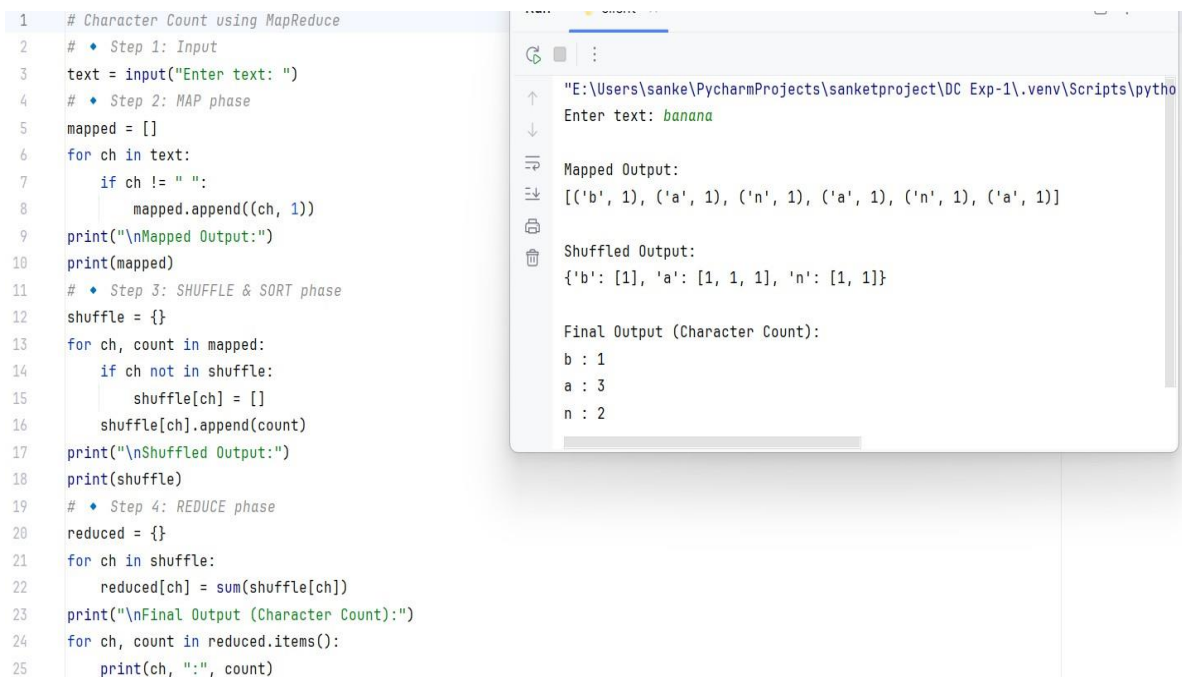
un server client

"E:\Users\sanke\PycharmProjects\sanketproject\DC Exp-1\.venv\Scripts\python.exe" "E:\Users\sanke\PycharmProjects\sanketproject\client.py"
Enter server URI: PYRO:obj_2db2fe8233a042f880757fc3ac2ba67f@localhost:62618
Enter first string: SAN
Enter second string: KET
Result: SANKET

Exp 3:

Character Count code:

```
# Character Count using MapReduce
# ♦ Step 1: Input
text = input("Enter text: ")
# ♦ Step 2: MAP phase
mapped = []
for ch in text:
    if ch != " ":
        mapped.append((ch, 1))
print("\nMapped Output:")
print(mapped)
# ♦ Step 3: SHUFFLE & SORT phase
shuffle = {}
for ch, count in mapped:
    if ch not in shuffle:
        shuffle[ch] = []
    shuffle[ch].append(count)
print("\nShuffled Output:")
print(shuffle)
# ♦ Step 4: REDUCE phase
reduced = {}
for ch in shuffle:
    reduced[ch] = sum(shuffle[ch])
print("\nFinal Output (Character Count):")
for ch, count in reduced.items():
    print(ch, ":", count)
```



The screenshot shows a code editor with the Python code for character counting using MapReduce. The code is divided into four steps: Step 1 (Input), Step 2 (MAP phase), Step 3 (SHUFFLE & SORT phase), and Step 4 (REDUCE phase). The output of the code is displayed in a separate window, showing the mapped output, shuffled output, and final output (character count).

```
1 # Character Count using MapReduce
2 # ♦ Step 1: Input
3 text = input("Enter text: ")
4 # ♦ Step 2: MAP phase
5 mapped = []
6 for ch in text:
7     if ch != " ":
8         mapped.append((ch, 1))
9 print("\nMapped Output:")
10 print(mapped)
11 # ♦ Step 3: SHUFFLE & SORT phase
12 shuffle = {}
13 for ch, count in mapped:
14     if ch not in shuffle:
15         shuffle[ch] = []
16     shuffle[ch].append(count)
17 print("\nShuffled Output:")
18 print(shuffle)
19 # ♦ Step 4: REDUCE phase
20 reduced = {}
21 for ch in shuffle:
22     reduced[ch] = sum(shuffle[ch])
23 print("\nFinal Output (Character Count):")
24 for ch, count in reduced.items():
25     print(ch, ":", count)
```

Output:

```
"E:\Users\sanke\PycharmProjects\sanketproject\DC Exp-1\.venv\Scripts\python.exe"
Enter text: banana

Mapped Output:
[('b', 1), ('a', 1), ('n', 1), ('a', 1), ('n', 1), ('a', 1)]

Shuffled Output:
{'b': [1], 'a': [1, 1, 1], 'n': [1, 1]}

Final Output (Character Count):
b : 1
a : 3
n : 2
```

Word Count code ;

Word Count using MapReduce (Manual Input)

◆ Step 1: Input

```
text = input("Enter text: ")
```

◆ Step 2: MAP phase

```
mapped = []
```

```
words = text.strip().split()
```

```
for word in words:
```

```
    mapped.append((word.lower(), 1))
```

```
print("\nMapped Output:")
```

```
print(mapped)
```

◆ Step 3: SHUFFLE & SORT phase

```
shuffle = {}
```

```
for word, count in mapped:
```

```
    if word not in shuffle:
```

```
        shuffle[word] = []
```

```
    shuffle[word].append(count)
```

```
print("\nShuffled Output:")
```

```
print(shuffle)
```

◆ Step 4: REDUCE phase

```
reduced = {}
```

```
for word in shuffle:
```

```
    reduced[word] = sum(shuffle[word])
```

```
print("\nFinal Output (Word Count):")
```

```
for word, count in reduced.items():
```

```
    print(word, ":", count)
```

```
1 # Word Count using MapReduce
2 # ♦ Step 1: Input
3 text = input("Enter text: ")
4 # ♦ Step 2: MAP phase
5 mapped = []
6 words = text.strip().split()
7 for word in words:
8     mapped.append((word.lower(), 1))
9 print("\nMapped Output:")
10 print(mapped)
11 # ♦ Step 3: SHUFFLE & SORT phase
12 shuffle = {}
13 for word, count in mapped:
14     if word not in shuffle:
15         shuffle[word] = []
16     shuffle[word].append(count)
17 print("\nShuffled Output:")
18 print(shuffle)
19 # ♦ Step 4: REDUCE phase
20 reduced = {}
21 for word in shuffle:
22     reduced[word] = sum(shuffle[word])
23 print("\nFinal Output (Word Count):")
24 for word, count in reduced.items():
25     print(word, ":", count)
```

Run client x

Final Output (Word Count):

apple : 3
banana : 2
mango : 1

Exp 4:

$A = \{x_1:0.2, x_2:0.5, x_3:0.7\}$

$B = \{x_1:0.3, x_2:0.6, x_3:0.4\}$

Union

$union = \{k: \max(A[k], B[k]) \text{ for } k \text{ in } A\}$

Intersection

$intersection = \{k: \min(A[k], B[k]) \text{ for } k \text{ in } A\}$

Complement

$comp_A = \{k: 1 - A[k] \text{ for } k \text{ in } A\}$

Difference A - B

$diff = \{k: \min(A[k], 1 - B[k]) \text{ for } k \text{ in } A\}$

print("Union:", union)

print("Intersection:", intersection)

print("Complement:", comp_A)

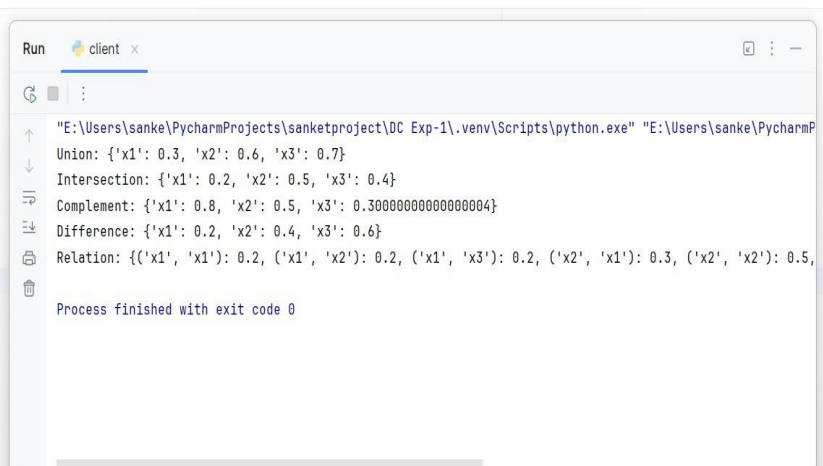
print("Difference:", diff)

Cartesian product

$relation = \{(a, b): \min(A[a], B[b]) \text{ for } a \text{ in } A \text{ for } b \text{ in } B\}$

print("Relation:", relation)

```
1 A = {'x1':0.2, 'x2':0.5, 'x3':0.7}
2 B = {'x1':0.3, 'x2':0.6, 'x3':0.4}
3
4 # Union
5 union = {k: max(A[k], B[k]) for k in A}
6
7 # Intersection
8 intersection = {k: min(A[k], B[k]) for k in A}
9
10 # Complement
11 comp_A = {k: 1 - A[k] for k in A}
12
13 # Difference A - B
14 diff = {k: min(A[k], 1 - B[k]) for k in A}
15
16 print("Union:", union)
17 print("Intersection:", intersection)
18 print("Complement:", comp_A)
19 print("Difference:", diff)
20
21 # Cartesian product
22 relation = {(a, b): min(A[a], B[b]) for a in A for b in B}
23 print("Relation:", relation)
```



The screenshot shows a PyCharm IDE window with a Python script on the left and a Run console on the right. The script defines two dictionaries A and B, and calculates their Union, Intersection, Complement, Difference, and Cartesian product. The Run console shows the output of the script, which matches the expected results for the given input.

```
Run client x
"E:\Users\sanke\PycharmProjects\sanketproject\DC Exp-1\.venv\Scripts\python.exe" "E:\Users\sanke\PycharmP
Union: {'x1': 0.3, 'x2': 0.6, 'x3': 0.7}
Intersection: {'x1': 0.2, 'x2': 0.5, 'x3': 0.4}
Complement: {'x1': 0.8, 'x2': 0.5, 'x3': 0.30000000000000004}
Difference: {'x1': 0.2, 'x2': 0.4, 'x3': 0.6}
Relation: {('x1', 'x1'): 0.2, ('x1', 'x2'): 0.2, ('x1', 'x3'): 0.2, ('x2', 'x1'): 0.3, ('x2', 'x2'): 0.5,
Process finished with exit code 0
```

EXP 5:

```
import random
import hashlib
servers = ["S1", "S2", "S3"]
weights = {"S1": 1, "S2": 2, "S3": 3}
# Sample requests (can be anything)
requests = ["R1", "R2", "R3", "R4", "R5", "R6", "R7"]

# ◆ 1. Round Robin
def round_robin(requests):
    print("\n--- Round Robin ---")
    for i, req in enumerate(requests):
        server = servers[i % len(servers)]
        print(f"{req} -> {server}")

# ◆ 2. Least Connections
def least_connections(requests):
    print("\n--- Least Connections ---")
    connections = {s: 0 for s in servers}
    for req in requests:
        server = min(connections, key=connections.get)
        print(f"{req} -> {server}")
        connections[server] += 1

# ◆ 3. Random
def random_lb(requests):
    print("\n--- Random ---")
    for req in requests:
        server = random.choice(servers)
        print(f"{req} -> {server}")

# ◆ 4. Weighted Round Robine
def weighted_round_robin(requests):
    print("\n--- Weighted Round Robin ---")
    weighted_list = []
    for s in servers:
        weighted_list.extend([s] * weights[s])
    for i, req in enumerate(requests):
        server = weighted_list[i % len(weighted_list)]
        print(f"{req} -> {server}")

# ◆ 5. IP Hash
def ip_hash(requests):
    print("\n--- IP Hash ---")
    for req in requests:
        ip = req # simulate request as IP
        hash_val = int(hashlib.md5(ip.encode()).hexdigest(), 16)
        server = servers[hash_val % len(servers)]
        print(f"{req} -> {server}")
```



```

# Run all
round_robin(requests)
least_connections(requests)
random_lb(requests)
weighted_round_robin(requests)
ip_hash(requests)

```

```

import random
import hashlib
servers = ["S1", "S2", "S3"]
weights = {"S1": 1, "S2": 2, "S3": 3}
# Sample requests (can be anything)
requests = ["R1", "R2", "R3", "R4", "R5", "R6", "R7"]

# ♦ 1. Round Robin
def round_robin(requests): 1usage new*
    print("\n--- Round Robin ---")
    for i, req in enumerate(requests):
        server = servers[i % len(servers)]
        print(f"{req} -> {server}")

# ♦ 2. Least Connections
def least_connections(requests): 1usage new*
    print("\n--- Least Connections ---")
    connections = {s: 0 for s in servers}
    for req in requests:
        server = min(connections, key=connections.get)
        print(f"{req} -> {server}")
        connections[server] += 1

# ♦ 3. Random
def random_lb(requests): 1usage new*
    print("\n--- Random ---")
    for req in requests:
        server = random.choice(servers)
        print(f"{req} -> {server}")

# ♦ 4. Weighted Round Robin
def weighted_round_robin(requests): 1usage new*
    print("\n--- Weighted Round Robin ---")
    weighted_list = []
    for s in servers:
        weighted_list.extend([s] * weights[s])

```

```

"E:\Users\sanke\PycharmProjects\sanketproject\DC Exp-1\.venv\Scr

--- Round Robin ---
R1 -> S1
R2 -> S2
R3 -> S3
R4 -> S1
R5 -> S2
R6 -> S3
R7 -> S1

--- Least Connections ---
R1 -> S1
R2 -> S2
R3 -> S3
R4 -> S1
R5 -> S2
R6 -> S3
R7 -> S1

--- Random ---
R1 -> S2
R2 -> S2
R3 -> S2
R4 -> S3
R5 -> S2
R6 -> S2
R7 -> S3

--- Weighted Round Robin ---
R1 -> S1
R2 -> S2

```

Exp 6:

```
import random
def fitness(x):
    return x**2
# ♦ Step 1: Initialization
population = [random.randint(1, 10) for _ in range(5)]
for generation in range(5):
    print(f"\nGeneration {generation+1}:", population)
    # ♦ Step 2: Fitness Evaluation
    population = sorted(population, key=fitness, reverse=True)
    # ♦ Step 3: Selection (best individuals)
    best = population[:2]
    # ♦ Step 4: Cloning
    clones = []
    for b in best:
        for _ in range(3):
            # ♦ Step 5: Mutation
            mutated = b + random.randint(-1, 1)
            clones.append(mutated)
    # ♦ Step 6: Selection from combined population
    combined = population + clones
    population = sorted(combined, key=fitness, reverse=True)[:5]
# Final best
best_solution = max(population, key=fitness)
print("\nBest solution:", best_solution)
```

```
import random
def fitness(x): 3 usages new *
    return x**2
# ♦ Step 1: Initialization
population = [random.randint(1, 10) for _ in range(5)]
for generation in range(5):
    print(f"\nGeneration {generation+1}:", population)
    # ♦ Step 2: Fitness Evaluation
    population = sorted(population, key=fitness, reverse=True)
    # ♦ Step 3: Selection (best individuals)
    best = population[:2]
    # ♦ Step 4: Cloning
    clones = []
    for b in best:
        for _ in range(3):
            # ♦ Step 5: Mutation
            mutated = b + random.randint(-1, 1)
            clones.append(mutated)
    # ♦ Step 6: Selection from combined population
    combined = population + clones
    population = sorted(combined, key=fitness, reverse=True)[:5]
# Final best
best_solution = max(population, key=fitness)
print("\nBest solution:", best_solution)
```

Run client x

```
"E:\Users\sanke\PycharmProjects\sanketprojec
Generation 1: [3, 7, 9, 1, 1]
Generation 2: [9, 9, 9, 8, 7]
Generation 3: [10, 10, 9, 9, 9]
Generation 4: [11, 11, 11, 11, 10]
Generation 5: [12, 12, 12, 11, 11]
Best solution: 13
Process finished with exit code 0
```

EXP 7:

```
import random
import math
# ◆ Sample dataset (features: vibration, crack_length)
# label: 0 = Undamaged, 1 = Damaged
data = [
    ([0.1, 0.2], 0),
    ([0.2, 0.1], 0),
    ([0.8, 0.9], 1),
    ([0.7, 0.8], 1),
    ([0.15, 0.25], 0),
    ([0.85, 0.95], 1)
]
# ◆ Distance function (Euclidean)
def distance(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))
# ◆ Step 1: Initialize antibodies (memory cells)
antibodies = []
for _ in range(4):
    features, label = random.choice(data)
    antibodies.append((features, label))
# ◆ Training (Clonal Selection inspired)
for iteration in range(5):
    new_antibodies = []
    for ab_features, ab_label in antibodies:
        # clone and mutate
        for _ in range(2):
            mutated = [x + random.uniform(-0.1, 0.1) for x in ab_features]
            new_antibodies.append((mutated, ab_label))
        # combine old + new
    antibodies.extend(new_antibodies)
    # select best antibodies (closest to training data)
    antibodies = sorted(
        antibodies,
        key=lambda ab: sum(distance(ab[0], d[0]) for d in data)
    )[:6]
# ◆ Classification function
def classify(sample):
    nearest = min(antibodies, key=lambda ab: distance(sample, ab[0]))
    return nearest[1]
# ◆ Testing + Evaluation
TP = TN = FP = FN = 0
print("\n--- Classification Results ---")
for features, actual in data:
    pred = classify(features)
    print(f'Input: {features} -> Predicted: {pred}, Actual: {actual}')
    if pred == 1 and actual == 1:
        TP += 1
    elif pred == 0 and actual == 0:
        TN += 1
```

```
elif pred == 1 and actual == 0:
```

```
    FP += 1
```

```
elif pred == 0 and actual == 1:
```

```
    FN += 1
```

```
# ◆ Metrics
```

```
accuracy = (TP + TN) / len(data)
```

```
sensitivity = TP / (TP + FN) if (TP + FN) != 0 else 0
```

```
specificity = TN / (TN + FP) if (TN + FP) != 0 else 0
```

```
print("\n--- Performance ---")
```

```
print("Accuracy:", round(accuracy, 2))
```

```
print("Sensitivity:", round(sensitivity, 2))
```

```
print("Specificity:", round(specificity, 2))
```

```
1 import random
2 import math
3 # ◆ Sample dataset (features: vibration, crack_length)
4 # label: 0 = Undamaged, 1 = Damaged
5 data = [
6     ([0.1, 0.2], 0),
7     ([0.2, 0.1], 0),
8     ([0.8, 0.9], 1),
9     ([0.7, 0.8], 1),
10    ([0.15, 0.25], 0),
11    ([0.85, 0.95], 1)
12 ]
13 # ◆ Distance function (Euclidean)
14 def distance(a, b): 2 usages new*
15     return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))
16 # ◆ Step 1: Initialize antibodies (memory cells)
17 antibodies = []
18 for _ in range(4):
19     features, label = random.choice(data)
20     antibodies.append((features, label))
21 # ◆ Training (Clonal Selection inspired)
22 for iteration in range(5):
23     new_antibodies = []
24     for ab_features, ab_label in antibodies:
25         # clone and mutate
26         for _ in range(2):
27             mutated = [x + random.uniform(-0.1, 0.1) for x in ab_features]
28             new_antibodies.append((mutated, ab_label))
29     # combine old + new
30     antibodies.extend(new_antibodies)
31     # select best antibodies (closest to training data)
32     antibodies = sorted(
```

```
"E:\Users\sanke\PycharmProjects\sanketproject\DC E

--- Classification Results ---
Input: [0.1, 0.2] -> Predicted: 0, Actual: 0
Input: [0.2, 0.1] -> Predicted: 0, Actual: 0
Input: [0.8, 0.9] -> Predicted: 0, Actual: 1
Input: [0.7, 0.8] -> Predicted: 0, Actual: 1
Input: [0.15, 0.25] -> Predicted: 0, Actual: 0
Input: [0.85, 0.95] -> Predicted: 0, Actual: 1

--- Performance ---
Accuracy: 0.5
Sensitivity: 0.0
Specificity: 1.0

Process finished with exit code 0
|
```

EXP 8:

```
# DEAP Distributed Evolutionary Algorithm (Single File)
from deap import base, creator, tools, algorithms
import random
import multiprocessing

# ♦ Step 1: Define Fitness and Individual
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

# ♦ Step 2: Toolbox setup
toolbox = base.Toolbox()
# Gene: random float between -10 and 10
toolbox.register("attr_float", random.uniform, -10, 10)
# Individual: 1 gene
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_float, 1)
# Population
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

# ♦ Step 3: Fitness function (maximize x^2)
def eval_func(individual):
    x = individual[0]
    return (x**2,)
toolbox.register("evaluate", eval_func)

# ♦ Step 4: Genetic operators
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

# ♦ Step 5: Main execution
def main():
    print("♦ Starting Distributed Evolutionary Algorithm...\n")
    # Create multiprocessing pool (for distributed execution)
    pool = multiprocessing.Pool()
    toolbox.register("map", pool.map)
    # Initialize population
    population = toolbox.population(n=10)
    print("Initial Population:")
    for ind in population:
        print(ind)
    print("\n♦ Running Evolution...\n")
    # Run evolutionary algorithm
    population, log = algorithms.eaSimple(
        population,
        toolbox,
        cxpb=0.5, # crossover probability
        mutpb=0.2, # mutation probability
        ngen=10, # number of generations
        verbose=True
    )
    # Get best individual
    best = tools.selBest(population, k=1)[0]
```

```
print("\n ■ Best Solution Found:")
print("Value of x:", best[0])
print("Maximum  $f(x)=x^2$ :", best.fitness.values[0])
# Close multiprocessing
pool.close()
pool.join()
# ◆ Run program
if __name__ == "__main__":
    main()
```

```

1  # DEAP Distributed Evolutionary Algorithm (Single File)
2  from deap import base, creator, tools, algorithms
3  import random
4  import multiprocessing
5  # ♦ Step 1: Define Fitness and Individual
6  creator.create( name: "FitnessMax", base.Fitness, weights=(1.0,))
7  creator.create( name: "Individual", list, fitness=creator.FitnessMax)
8  # ♦ Step 2: Toolbox setup
9  toolbox = base.Toolbox()
10 # Gene: random float between -10 and 10
11 toolbox.register( alias: "attr_float", random.uniform, *args: -10, 10)
12 # Individual: 1 gene
13 toolbox.register( alias: "individual", tools.initRepeat, *args: creator.Individual, toolbox.attr_float, 1)
14 # Population
15 toolbox.register( alias: "population", tools.initRepeat, *args: list, toolbox.individual)
16 # ♦ Step 3: Fitness function (maximize x^2)
17 def eval_func(individual): 1usage new *
18     x = individual[0]
19     return (x**2,)
20 toolbox.register( alias: "evaluate", eval_func)
21 # ♦ Step 4: Genetic operators
22 toolbox.register( alias: "mate", tools.cxBlend, alpha=0.5)
23 toolbox.register( alias: "mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
24 toolbox.register( alias: "select", tools.selTournament, tournsize=3)
25 # ♦ Step 5: Main execution
26 def main(): 1usage new *
27     print(" ♦ Starting Distributed Evolutionary Algorithm...\n")
28     # Create multiprocessing pool (for distributed execution)
29     pool = multiprocessing.Pool()
30     toolbox.register( alias: "map", pool.map)

```

```

26 def main(): 1usage new *
29     pool = multiprocessing.Pool()
30     toolbox.register( alias: "map", pool.map)
31     # Initialize population
32     population = toolbox.population(n=10)
33     print("Initial Population:")
34     for ind in population:
35         print(ind)
36     print("\n ♦ Running Evolution...\n")
37     # Run evolutionary algorithm
38     population, log = algorithms.eaSimple(
39         population,
40         toolbox,
41         cxpb=0.5, # crossover probability
42         mutpb=0.2, # mutation probability
43         ngen=10, # number of generations
44         verbose=True
45     )
46     # Get best individual
47     best = tools.selBest(population, k=1)[0]
48     print("\n 🟢 Best Solution Found:")
49     print("Value of x:", best[0])
50     print("Maximum f(x)=x^2:", best.fitness.values[0])
51     # Close multiprocessing
52     pool.close()
53     pool.join()
54 # ♦ Run program
55 if __name__ == "__main__":
56     main()

```

hon Packages

"E:\Users\sanke\PycharmProjects\sanketproject\DC Exp-1\venv"
♦ Starting Distributed Evolutionary Algorithm...

Initial Population:

[4.682029863758894]
[1.498048764823249]
[-3.665365547402497]
[-9.951003428844453]
[-3.6421331818387674]
[-9.45075008493108]
[4.893587783508163]
[8.166761158802398]
[0.49078332848779027]
[9.149606187391658]

♦ Running Evolution...

gen	nevals
0	10
1	8
2	7
3	8
4	7
5	6
6	6
7	7
8	7
9	6
10	4

🟢 Best Solution Found:
Value of x: -46.64025945393004
Maximum f(x)=x^2: 2175.313801929911

Exp 9:

```
import java.rmi.*;
```

```
import java.rmi.registry.*;
```

```
import java.rmi.server.*;
```

```
import java.util.*;
```

```
// ♦ Remote Interface
```

```
interface HotelService extends Remote {
```

```
    String bookRoom(String guestName) throws RemoteException;
```

```
    String cancelBooking(String guestName) throws RemoteException;
```

```
}
```

```
// ♦ Implementation
```

```
class HotelServiceImpl extends UnicastRemoteObject implements HotelService {
```

```
    private HashMap<String, String> bookings;
```

```
    protected HotelServiceImpl() throws RemoteException {
```

```
        bookings = new HashMap<>();
```

```
    }
```

```
    public synchronized String bookRoom(String guestName) throws RemoteException {
```

```
        if (bookings.containsKey(guestName)) {
```

```
            return "Room already booked for " + guestName;
```

```
        }
```

```
        bookings.put(guestName, "Booked");
```

```
        return "Room successfully booked for " + guestName;
```

```
    }
```

```
    public synchronized String cancelBooking(String guestName) throws RemoteException {
```

```
        if (!bookings.containsKey(guestName)) {
```

```
            return "No booking found for " + guestName;
```

```
        }
```

```
        bookings.remove(guestName);
```

```
        return "Booking cancelled for " + guestName;
```

```
    }
```

```
}
```



```
// ◆ Main Class (Server + Client)

public class HotelSystem {

    public static void main(String[] args) {

        try {

            // Start RMI registry inside code
            LocateRegistry.createRegistry(1099);

            // Create object
            HotelServiceImpl obj = new HotelServiceImpl();

            // Bind service
            Naming.rebind("rmi://localhost/HotelService", obj);

            System.out.println(" ■ Server is running...");

            // ◆ Client Part
            HotelService stub = (HotelService) Naming.lookup("rmi://localhost/HotelService");
            Scanner sc = new Scanner(System.in);

            while (true) {

                System.out.println("\n1. Book Room");
                System.out.println("2. Cancel Booking");
                System.out.println("3. Exit");
                System.out.print("Enter choice: ");
                int choice = sc.nextInt();
                sc.nextLine();

                if (choice == 1) {

                    System.out.print("Enter guest name: ");
                    String name = sc.nextLine();
                    System.out.println(stub.bookRoom(name));

                }

                else if (choice == 2) {

                    System.out.print("Enter guest name: ");
                    String name = sc.nextLine();
                    System.out.println(stub.cancelBooking(name));

                }

            }

        }

    }

}
```

```

    }
    else {
        break;
    }
}

} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

```

33 // ♦ Main Class (Server + Client)
34 public class HotelSystem {
35     Run main | Debug main | Run | Debug
36     public static void main(String[] args) {
37         try {
38             // Start RMI registry inside code
39             LocateRegistry.createRegistry(port: 1099);
40             // Create object
41             HotelServiceImpl obj = new HotelServiceImpl();
42             // Bind service
43             Naming.rebind(name: "rmi://localhost/HotelService", obj);
44             System.out.println(x: "✅ Server is running...");
45             // ♦ Client Part
46             HotelService stub = (HotelService) Naming.lookup(name: "rmi://localhost/HotelService");
47             Scanner sc = new Scanner(System.in);
48             while (true) {
49                 System.out.println(x: "\n1. Book Room");
50                 System.out.println(x: "2. Cancel Booking");
51                 System.out.println(x: "3. Exit");
52                 System.out.print(s: "Enter choice: ");
53                 int choice = sc.nextInt();
54                 sc.nextLine();
55                 if (choice == 1) {
56                     System.out.print(s: "Enter guest name: ");
57                     String name = sc.nextLine();
58                     System.out.println(stub.bookRoom(name));
59                 }
60                 else if (choice == 2) {
59                     System.out.print(s: "Enter guest name: ");
60

```

```

1  import java.rmi.*;
2  import java.rmi.registry.*;
3  import java.rmi.server.*;
4  import java.util.*;
5  // ♦ Remote Interface
6  interface HotelService extends Remote {
7      String bookRoom(String guestName) throws RemoteException;
8      String cancelBooking(String guestName) throws RemoteException;
9  } package <unnamed>
10 // ♦ Implementation
11 class HotelServiceImpl extends UnicastRemoteObject implements HotelService {
12
13     private HashMap<String, String> bookings;
14
15     protected HotelServiceImpl() throws RemoteException {
16         bookings = new HashMap<>();
17     }
18     public synchronized String bookRoom(String guestName) throws RemoteException {
19         if (bookings.containsKey(guestName)) {
20             return "Room already booked for " + guestName;
21         }
22         bookings.put(guestName, value: "Booked");
23         return "Room successfully booked for " + guestName;
24     }
25     public synchronized String cancelBooking(String guestName) throws RemoteException {
26         if (!bookings.containsKey(guestName)) {
27             return "No booking found for " + guestName;
28         }
29         bookings.remove(guestName);
30         return "Booking cancelled for " + guestName;
31     }
32 }

```

```

    public static void main(String[] args) {
        System.out.println(x: "3. Exit");
        System.out.print(s: "Enter choice: ");
        int choice = sc.nextInt();
        sc.nextLine();
        if (choice == 1) {
            System.out.print(s: "Enter guest name: ");
            String name = sc.nextLine();
            System.out.println(stub.bookRoom(name));
        }
        else if (choice == 2) {
            System.out.print(s: "Enter guest name: ");
            String name = sc.nextLine();
            System.out.println(stub.cancelBooking(name));
        }
        else {
            break;
        }
    }
} catch (Exception e) {
    e.printStackTrace();
}
}

```

1. Book Room
2. Cancel Booking
3. Exit

Enter choice: 1

Enter guest name: Sanket

Room successfully booked for Sanket

Enter choice: 2

Enter guest name: Sanket

Booking cancelled for Sanket

Exp 10:

```
import numpy as np
import matplotlib.pyplot as plt
import random
# ◆ Number of cities
num_cities = 5
# ◆ Generate random city coordinates
cities = np.random.rand(num_cities, 2)
# ◆ Distance matrix
def distance(a, b):
    return np.linalg.norm(a - b)
dist_matrix = np.zeros((num_cities, num_cities))
for i in range(num_cities):
    for j in range(num_cities):
        dist_matrix[i][j] = distance(cities[i], cities[j])
# ◆ ACO parameters
num_ants = 10
num_iterations = 50
alpha = 1    # pheromone importance
beta = 2     # distance importance
evaporation = 0.5
# ◆ Initialize pheromone
pheromone = np.ones((num_cities, num_cities))
# ◆ Function to choose next city
def choose_next_city(current, visited):
    probs = []
    for j in range(num_cities):
        if j not in visited:
            tau = pheromone[current][j] ** alpha
            eta = (1.0 / dist_matrix[current][j]) ** beta
            probs.append((j, tau * eta))

    total = sum(p for _, p in probs)
    probs = [(j, p / total) for j, p in probs]
    r = random.random()
    cumulative = 0
    for j, p in probs:
        cumulative += p
        if r <= cumulative:
            return j
# ◆ Main ACO loop
best_path = None
best_length = float('inf')
for iteration in range(num_iterations):
    all_paths = []
    all_lengths = []
    for ant in range(num_ants):
        path = [random.randint(0, num_cities - 1)]
        while len(path) < num_cities:
```

```

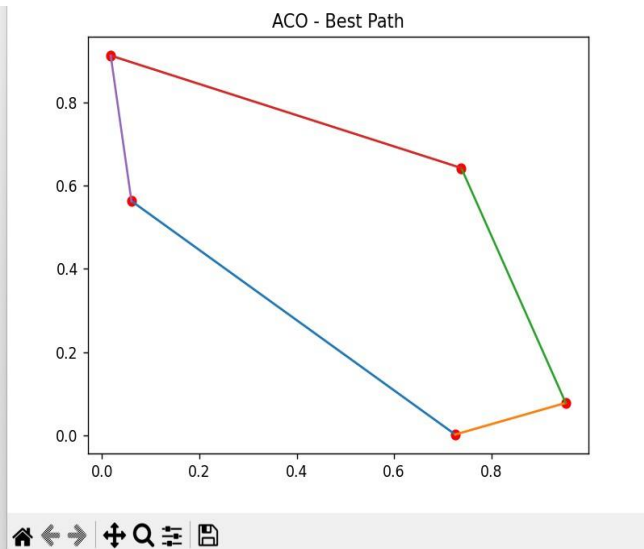
        next_city = choose_next_city(path[-1], path)
        path.append(next_city)
    path.append(path[0]) # return to start
    length = sum(dist_matrix[path[i]][path[i+1]] for i in range(len(path)-1))
    all_paths.append(path)
    all_lengths.append(length)
    if length < best_length:
        best_length = length
        best_path = path
# ◆ Evaporation
pheromone *= (1 - evaporation)
# ◆ Update pheromone
for path, length in zip(all_paths, all_lengths):
    for i in range(len(path)-1):
        pheromone[path[i]][path[i+1]] += 1.0 / length
print(f"Iteration {iteration+1}: Best Length = {best_length:.4f}")
# ◆ Plot best path
x = cities[:, 0]
y = cities[:, 1]
plt.scatter(x, y, color='red')
for i in range(len(best_path)-1):
    c1 = cities[best_path[i]]
    c2 = cities[best_path[i+1]]
    plt.plot([c1[0], c2[0]], [c1[1], c2[1]])
plt.title("ACO - Best Path")
plt.show()
print("\nBest Path:", best_path)
print("Shortest Distance:", best_length)

```

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import random
4 # ♦ Number of cities
5 num_cities = 5
6 # ♦ Generate random city coordinates
7 cities = np.random.rand(num_cities, 2)
8 # ♦ Distance matrix
9 def distance(a, b): 1usage new*
10     return np.linalg.norm(a - b)
11 dist_matrix = np.zeros((num_cities, num_cities))
12 for i in range(num_cities):
13     for j in range(num_cities):
14         dist_matrix[i][j] = distance(cities[i], cities[j])
15 # ♦ ACO parameters
16 num_ants = 10
17 num_iterations = 50
18 alpha = 1 # pheromone importance
19 beta = 2 # distance importance
20 evaporation = 0.5
21 # ♦ Initialize pheromone
22 pheromone = np.ones((num_cities, num_cities))
23 # ♦ Function to choose next city
24 def choose_next_city(current, visited): 1usage new*
25     probs = []
26     for j in range(num_cities):
27         if j not in visited:
28             tau = pheromone[current][j] ** alpha

```



```

24 def choose_next_city(current, visited): 1usage new*
26     for j in range(num_cities):
27         if j not in visited:
28             tau = pheromone[current][j] ** alpha
29             eta = (1.0 / dist_matrix[current][j]) ** beta
30             probs.append((j, tau * eta))
31     total = sum(p for _, p in probs)
32     probs = [(j, p / total) for j, p in probs]
33     r = random.random()
34     cumulative = 0
35     for j, p in probs:
36         cumulative += p
37         if r <= cumulative:
38             return j
39 # ♦ Main ACO loop
40 best_path = None
41 best_length = float('inf')
42 for iteration in range(num_iterations):
43     all_paths = []
44     all_lengths = []
45     for ant in range(num_ants):
46         path = [random.randint(0, num_cities - 1)]
47         while len(path) < num_cities:
48             next_city = choose_next_city(path[-1], path)
49             path.append(next_city)
50         path.append(path[0]) # return to start
51         length = sum(dist_matrix[path[i]][path[i+1]] for i in range(len(path)-1))
52         all_paths.append(path)

```

```

49         path.append(next_city)
50     path.append(path[0]) # return to start
51     length = sum(dist_matrix[path[i]][path[i+1]] for i in range(len(path)-1))
52     all_paths.append(path)
53     all_lengths.append(length)
54     if length < best_length:
55         best_length = length
56         best_path = path
57     # ♦ Evaporation
58     pheromone *= (1 - evaporation)
59     # ♦ Update pheromone
60     for path, length in zip(all_paths, all_lengths):
61         for i in range(len(path)-1):
62             pheromone[path[i]][path[i+1]] += 1.0 / length
63     print(f"Iteration {iteration+1}: Best Length = {best_length:.4f}")
64     # ♦ Plot best path
65     x = cities[:, 0]
66     y = cities[:, 1]
67     plt.scatter(x, y, color='red')
68     for i in range(len(best_path)-1):
69         c1 = cities[best_path[i]]
70         c2 = cities[best_path[i+1]]
71         plt.plot(*args: [c1[0], c2[0]], [c1[1], c2[1]])
72     plt.title("ACO - Best Path")
73     plt.show()
74     print("\nBest Path:", best_path)
75     print("Shortest Distance:", best_length)

```

Run client

↺

⏏

⋮

↑

↓

↶

↷

🖨

🗑

```

Iteration 45: Best Length = 2.8359
Iteration 46: Best Length = 2.8359
Iteration 47: Best Length = 2.8359
Iteration 48: Best Length = 2.8359
Iteration 49: Best Length = 2.8359
Iteration 50: Best Length = 2.8359

Best Path: [1, 0, 3, 4, 2, 1]
Shortest Distance: 2.835884131520456

```